

# Architecture Documentation — Collaborative Canvas

## 1. Overview

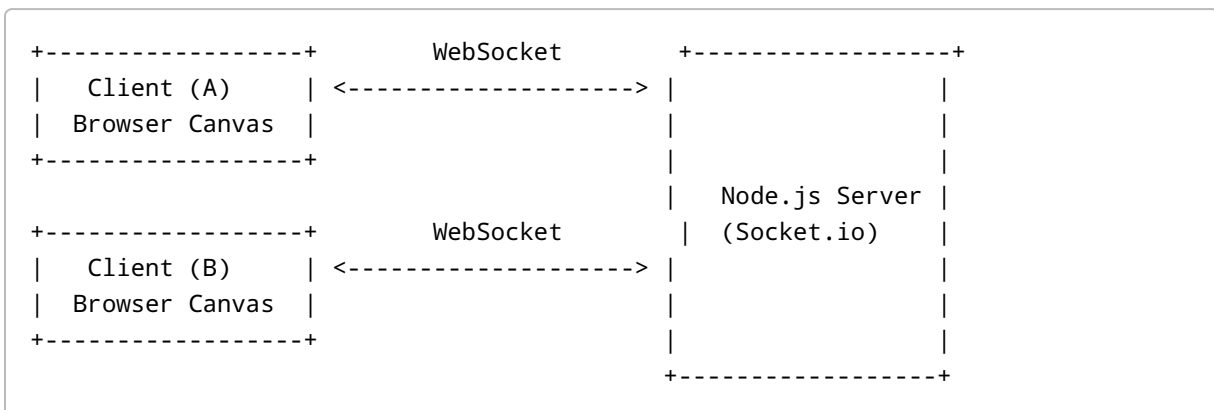
The **Collaborative Canvas** is a real-time, multi-user drawing application that allows multiple clients to draw simultaneously on a shared HTML5 canvas. The system is built using a **client-server architecture** where:

- The **client** handles rendering, user input, and UI interactions.
- The **server** acts as the single source of truth for shared drawing state and synchronizes all connected clients using WebSockets.

The design prioritizes: - Low-latency real-time updates - Deterministic state synchronization - Clear separation of responsibilities

---

## 2. High-Level Architecture



- All clients connect to a **single backend server**.
  - Clients never communicate directly with each other.
  - The server maintains global drawing state and broadcasts updates.
- 

## 3. Client-Side Architecture

### 3.1 Responsibilities

The client is responsible for: - Capturing mouse/touch input - Rendering strokes on the canvas - Sending drawing events to the server - Rendering remote users' strokes - Displaying ghost cursors - Triggering undo actions

### 3.2 Client Folder Structure

```
client/
├─ index.html      # UI structure
├─ style.css       # Layout and styling
├─ main.js         # App bootstrap & event wiring
├─ canvas.js       # Canvas drawing & rendering logic
```

### 3.3 Canvas Rendering Model

- The canvas uses the **HTML5 2D rendering context** (`getContext('2d')`).
- Drawing is performed using **path segments** (start → end points).
- Mouse coordinates are normalized to canvas coordinates to handle CSS scaling.

Each stroke is composed of:

```
{
  id: string,
  userId: string,
  points: [{ x, y }, ...],
  color: string,
  width: number
}
```

### 3.4 Event Flow (Client)

1. User presses mouse → start stroke
2. Mouse moves → generate stroke segments
3. Stroke segments are:
4. Drawn locally immediately (optimistic rendering)
5. Emitted to server via WebSocket
6. Client listens for:
7. `draw` events (other users)
8. `state:init` (full redraw for late joiners)
9. `state:update` (undo or sync events)

## 4. Server-Side Architecture

### 4.1 Responsibilities

The server is responsible for: - Managing WebSocket connections - Assigning unique user IDs - Maintaining global drawing history - Broadcasting drawing updates - Handling undo requests safely

### 4.2 Server Folder Structure

```
server/  
├─ server.js      # HTTP + WebSocket server  
├─ state-manager.js # Global drawing state & undo logic
```

---

### 4.3 Server as Source of Truth

The server maintains a **canonical history of strokes**:

```
strokes = [ stroke1, stroke2, stroke3, ... ]
```

Clients are considered **stateless renderers**: - They can disconnect and reconnect at any time - On connection, server sends full stroke history

---

### 4.4 Undo Strategy (Per-User Undo)

Undo is implemented using **ownership-based stroke removal**:

- Each stroke is tagged with `userId`
- When a user clicks Undo:
  - Server finds the latest stroke by that user
  - Removes only that stroke
  - Broadcasts updated stroke list

This avoids: - Deleting other users' work - Layer conflicts - Canvas flickering

---

## 5. WebSocket Communication Protocol

### 5.1 Core Events

| Event Name | Direction       | Purpose             |
|------------|-----------------|---------------------|
| draw       | Client → Server | Send stroke segment |
| draw       | Server → Client | Broadcast stroke    |
| undo       | Client → Server | Request undo        |
| state:init | Server → Client | Full canvas state   |
| cursor     | Client → Server | Cursor position     |
| cursor     | Server → Client | Ghost cursors       |

### 5.2 Data Serialization

All events use lightweight JSON payloads:

```
{
  "type": "draw",
  "start": { "x": 100, "y": 120 },
  "end": { "x": 110, "y": 125 },
  "color": "#000",
  "width": 4
}
```

This minimizes bandwidth and improves responsiveness.

## 6. State Synchronization Model

### Key Principles

- **Server-authoritative state**
- **Event-based updates**
- **Idempotent redraws**

When state changes (undo or new user join): 1. Server sends full stroke history 2. Clients clear canvas 3. Clients redraw strokes deterministically

## 7. Conflict Resolution

The system avoids conflicts by design:

- No shared mutable canvas state between clients
- No pixel-level synchronization
- Strokes are atomic operations

If two users draw simultaneously: - Server serializes events in arrival order - All clients replay strokes in the same order

Result: **eventual consistency without flicker.**

---

## 8. Scalability Considerations

Current design supports: - Dozens of concurrent users per room

Possible future improvements: - Room-based namespaces - Stroke compression - Snapshotting old history - Redis-backed state storage

---

## 9. Technology Stack

- **Frontend:** HTML5, Vanilla JavaScript, Canvas API
  - **Backend:** Node.js, Express, Socket.io
  - **Protocol:** WebSocket
  - **Runtime:** Node.js v18+
- 

## 10. Summary

This architecture cleanly separates rendering, networking, and state management while ensuring:

- Smooth real-time collaboration
- Predictable undo behavior
- Easy extensibility
- Clear debugging paths

The system is intentionally simple, deterministic, and robust—ideal for learning real-time collaborative system design.

---

✨ *End of Architecture Document*